

模块七: 智能体开发实战

第二十五讲: 多模态感知——为智能体装上“眼睛”

[解释]

本讲概览

本讲是模块七的开篇, 主要做两件事:

1. 把上周用 `requests` 手写的 HTTP 调用, 升级为更简洁的 OpenAI SDK。SDK 是一套封装好的工具库, 不需要再手动拼 HTTP 请求。
2. 从纯文本输入扩展到图像输入。也就是让程序不仅能处理文字, 还能“看懂”图片——这就是所谓的“多模态”(同时处理文本和图像两种信息形式)。

本讲的实战环节复用了上周学过的 FastAPI 和 Web 前端基础。

本讲教学目标

完成本讲后，大家应能够：

- **[识记层]** 说出智能体的三个核心环节（感知、推理、行动），并举例说明每个环节在程序中对应什么操作。
- **[理解层]** 解释为什么我们要从上周的手动拼接 HTTP 请求，升级到一套更简洁的工具库来调用大模型。
- **[应用层]** 编写一个 Python 程序，让大模型“看懂”一张本地图片并返回描述。
- **[应用层]** 在上周 FastAPI 的基础上，**搭建**一个能上传图片并获取 AI 分析结果的网页应用。
- **[分析层]** 论证为什么调用大模型的密钥必须放在服务器端，而不能写在网页代码里。
- **[评价层]** 面对不同场景（本地私密图片 vs. 网络上的公开图片），**判断**应该用哪种方式把图片传给大模型，并**说明理由**。

[解释]

布鲁姆认知目标阶梯

本页的教学目标按照布鲁姆认知分类法分层排列，从低到高依次是：识记（能说出是什么）→ 理解（能解释为什么）→ 应用（能动手写代码）→ 分析（能论证设计决策）→ 评价（能在多个方案中做出判断）。

这意味着本讲不仅要求记住概念，还要求能够独立写出代码、并且能论证架构决策的合理性。其中最高层的“评价”目标——判断图片传输方案——是本讲的最高层级能力目标。

模块七 学习路线图: 4 节课点亮智能体技能树

我们将用 4 节课的时间，从感知到行动，逐步构建一个完整的 AI 智能体：

- **L25: 多模态感知** (本节课)
升级 SDK 工具链，为程序装上"眼睛" (VLM 视觉理解)。
- **L26: 多模型 Pipeline** (下节课)
让多个 AI 模型在流水线上接力协作，完成复杂任务。
- **L27: Chrome 插件开发** (第三节)
将 AI 能力嵌入浏览器，开发离用户最近的智能体。
- **L28: RAG 知识库学伴** (第四节)
感受智能体有无"记忆"时的能力差异 + 黑客松正式启动。

通关目标

本模块结束时，每位同学将产生一个 **MVP（最小可行性产品）** 的原型代码，在第 8 周的 Demo Day 上展示作品。

能力叠加逻辑

每一节课都在前一节的基础上叠加新能力。L25 的感知 + L26 的推理 + L27 的行动 + L28 的记忆 = 一个完整的智能体。

[解释]

模块七的知识叠加逻辑

本页展示了模块七 4 节课的整体结构。每节课都在前一节的基础上叠加新能力：L25 解决"感知"（视觉）、L26 解决"推理"（多模型协作）、L27 解决"行动"（浏览器插件）、L28 解决"记忆"（RAG 知识库）。

这种递进式设计意味着：如果缺席了某一节课，后续内容可能会缺少前置基础。建议按顺序学习，并确保每节课的代码都能跑通后再进入下一节。

一、智能体的本质： 感知-推理-行动闭环

Perception - Reasoning - Action

[解释]

章节定位：什么是智能体

本章从概念层面定义智能体（Agent）。核心观点是：智能体不等于大模型聊天，而是一个能自主完成感知环境、推理决策、执行行动闭环的程序。理解这个定义是后续所有技术实现的前提。

什么是真正的智能体 (Agent)?

在之前的课程中，我们已经掌握了如何用代码调度大模型（LLM）。但这只是智能体的“大脑”。

智能体的核心本质，在于它能够与“真实环境”进行闭环交互。

一个完整的智能体（Agent）必然包含三大核心环节：

1. **感知 (Perception)**：从环境中获取多模态信息（如：看懂屏幕图片、抓取实时网页数据）。
2. **推理 (Reasoning)**：利用 LLM 的逻辑能力，基于感知到的信息进行思考、规划和决策。
3. **行动 (Action)**：通过执行代码或调用外部 API，对环境施加影响（如：自动提交表单、发送指令）。

从单向调用到自主闭环

普通的程序是“人类下发指令，模型返回文本”的单向请求-响应模式。而智能体的本质是“**感知-推理-行动**”的无限循环。它可以自主观察环境变化，动态调整策略，直到最终完成任务。

[解释]

智能体的三要素定义

本页提出了智能体最核心的概念：感知-推理-行动闭环。普通的 AI 聊天工具只是“一问一答”的单向模式，而智能体能够自主观察环境、做出决策、然后执行操作，形成不断循环的闭环。

三个要素缺一不可：缺少感知，程序无法获取外界信息；缺少推理，程序无法做出判断；缺少行动，程序无法影响环境。本周 4 节课就是逐步补齐这三个环节。

智能应用 vs. 智能体：单向响应与自主闭环的本质区别

学术界定义：

在经典人工智能教材《Artificial Intelligence: A Modern Approach》(Russell & Norvig) 中，智能体被严格定义为：“能够通过传感器（Sensors）感知环境，并通过执行器（Actuators）对环境采取行动的实体。”

缺少感知，它只是个盲盒；缺少行动，它只是个计算器。只有形成“感知-推理-行动”的物理或软件闭环，才能被称为“体”。

本周的拼图：

今天（L25），我们将为代码装上“眼睛”（多模态视觉），补齐智能体最缺乏的“感知”能力。随后，我们将用真实的代码，将这三个环节串联成一个完整的闭环！

[解释]

智能体 (Agent) 与普通 AI 应用的区别

本页的核心观点是：光有一个聊天机器人不算智能体。智能体必须能形成“感知-推理-行动”的闭环。

举个例子：网页版 ChatGPT 只能在聊天框里回复文字，它无法主动读取文件、也无法自己执行代码，所以它只是一个 AI 应用。而 Qwen Code 这样的编程助手能自己读文件、写代码、运行程序，它就是一个智能体。

这个区分很重要：本周的 4 节课就是在逐步补齐智能体的三个环节。今天补“感知”（让程序看图），后续补“推理”和“行动”。

二、从 requests 到 OpenAI SDK

从底层协议组装到 SDK 抽象封装

[解释]

章节定位：工具升级

本章讲解从手动拼装 HTTP 请求（requests 库）到使用 OpenAI SDK 的升级过程。SDK 屏蔽了底层通信细节，让调用大模型变得和调用普通函数一样简单。这是后续所有代码实战的技术基础。

回顾：HTTP 请求的底层组装

基于 requests 的原始调用

上周，我们通过手动拼装 HTTP 协议的请求头和数据包来完成调用：

```
import requests
import os

url = "https://api.siliconflow.cn/v1/chat/completions"
headers = {
    "Authorization": f"Bearer {os.environ['SILICONFLOW_KEY']}",
    "Content-Type": "application/json"
}
payload = {
    "model": "deepseek-ai/DeepSeek-V4-Flash",
    "messages": [{"role": "user", "content": "你好"}]
}

response = requests.post(url, json=payload, headers=headers)
print(response.json()["choices"][0]["message"]["content"])
```

这种模式的工程缺陷

1. 代码噪音过大：网络协议细节（headers、数据封包）掩盖了真正的核心——Prompt 业务逻辑，导致 AI 生成的代码臃肿，初学者难以抓取重点。
2. 高级功能扩展困难：在后续开发“流式对话”、“工具调用”等高级智能体功能时，如果继续让 AI 用底层协议手写，会生成极其复杂且难以维护的通信代码。

架构演进：我们需要一个更高级的抽象层（SDK），将底层网络逻辑与结构解析进行封装。

[解释]

requests 库的角色与局限

`requests` 是 Python 中最常用的 HTTP 请求库，但在人机协同场景下，要求 AI 助手用 `requests` 去发请求，会生成大量底层协议代码，掩盖了真正的 Prompt 逻辑。此外，在后续开发工具调用等高级功能时，底层代码会变得极其臃肿。

理解这页展示的两个缺陷（代码噪音过大、扩展困难），才能理解下一页为什么要升级到 SDK 提升抽象层级。

升级：引入 OpenAI SDK (标准化调用)

标准化的面向对象调用

`openai` 库已成为全球大模型接口的事实标准。不管是调用 DeepSeek 还是通义千问，只要它兼容此标准，这套代码就可以通用。

```
from openai import OpenAI
import os

# 1. 初始化客户端（指定自定义服务商）
client = OpenAI(
    api_key=os.environ["SILICONFLOW_KEY"],
    base_url="https://api.siliconflow.cn/v1"
)

# 2. 发起请求（直接传参数，无需处理底层 HTTP）
response = client.chat.completions.create(
    model="deepseek-ai/DeepSeek-V4-Flash",
    messages=[{"role": "user", "content": "你好"}]
)

# 3. 获取结果（属性访问）
print(response.choices[0].message.content)
```

SDK (工具包) 的核心优势

1. **抽象层跃升**：屏蔽所有底层网络细节，AI 生成的代码变得极其干净，使非计算机专业同学只需关注“模型配置”与“提示词设计”。
2. **一行开启高级能力**：SDK 已将复杂的智能体通信协议标准化。未来只需让 AI 助手增加一个参数（如 `tools=[...]`），就能给程序装上眼睛和手脚。
3. **一行切换百模**：只改 `api_key` 和 `base_url`，逻辑代码一行不改，瞬间切换至全球任意兼容生态。
4. **科研场景通用**：在涉及知识产权保护的科研项目中，通常需要本地部署大模型（Ollama / vLLM）。这些引擎同样暴露 OpenAI 兼容接口，只需将 `base_url` 改为 `http://localhost:11434/v1`，代码无需任何修改。

架构优势：无论云端商业平台还是实验室本地部署，学会这个库，就能调用几乎所有兼容大模型。

[解释]

SDK (Software Development Kit)

SDK 就是一套封装好的高级工具库。在人机协同场景下，要求 AI 助手用 `requests` 会生成大量干扰视线的网络通信代码。升级到 SDK，屏蔽了底层细节，使生成的代码紧凑且专注于业务本身（Prompt），并且为后续一行代码开启复杂智能体功能打下基础。

本地部署与知识产权保护

在科研场景中，实验数据（如医学影像、专利文献）往往不允许上传至第三方云端。此时需要在本地 GPU 服务器上部署开源大模型。主流的本地推理引擎（Ollama、vLLM、SGLang 等）均默认提供 OpenAI 兼容的 HTTP 接口。这意味着同一套 `openai` SDK 代码，只需修改 `base_url` 指向内网地址（如 `http://192.168.1.100:8000/v1`），即可在不泄露任何数据的前提下完成推理任务。这是该 SDK 在学术界被广泛采用的重要原因。

快速检验：SDK 的核心价值

情境题：假设你在实验室的 GPU 服务器上用 Ollama 部署了一个开源模型，现在想把刚才调用硅基流动的代码切换过去。需要修改哪几行代码？

答案：只需修改两处——`base_url` 改为 `http://localhost:11434/v1`，`api_key` 改为本地任意字符串。其余代码一字不动。这就是 SDK 统一接口的核心价值。

[解释]

SDK 统一接口的工程价值

本页是一个自测题。答案揭示了 OpenAI SDK 最重要的工程价值：接口统一。无论对接云端商业平台（如硅基流动）还是实验室本地部署的开源模型（如 Ollama），只需修改 `base_url` 和 `api_key` 两个参数，其余代码完全不变。

这个特性在科研场景中尤其重要：开发时可以用便宜的云端 API 调试，上线时切换到本地服务器保护数据隐私，不需要重写任何业务逻辑。

三、多模态：为代码 装上“眼睛”

VLM（视觉语言大模型）让程序理解图像

[解释]

章节定位：感知能力扩展

本章是核心实战部分。从纯文本输入扩展到图片输入（多模态），让程序具备视觉感知能力。技术关键是把图片编码为 Base64 文本，塞进 API 请求的 content 数组中。

真实产品：程序“看懂”图片能做什么？

Be My Eyes —— 让视障用户用 AI “看见”世界

Be My Eyes 是一款免费手机 App，服务全球数十万视障用户。

2023 年接入 GPT-4 视觉能力后，用户只需用手机拍一张照片（如冰箱里的食材、药品说明书、路牌），AI 就能用自然语言描述图片内容，甚至根据食材推荐菜谱。

技术链路与本讲完全一致：

手机拍照 → 图片编码传输 → VLM 分析
→ 返回文字描述

教学启示：大家今天课上写的代码，和 Be My Eyes 这款服务数十万人的产品，核心技术链路是一样的。区别只在于前端载体（手机 App vs. 网页）和模型规模。

思考：这个产品为什么必须用“拍照 + AI”，而不能用传统 OCR？因为用户需要的不是“识别文字”，而是“理解场景”——这正是 VLM 相比传统计算机视觉的核心优势。

[解释]

Be My Eyes 案例的教学意义

本页展示了一个真实的商业产品，它的核心技术链路和本讲要写的代码完全一致：拍照 → 编码传输 → VLM 分析 → 返回文字。这说明本讲学的技术不是玩具，而是已经在服务数十万用户的工业级技术。

页面中提到的 VLM 与传统 OCR 的区别也值得注意：OCR 只能识别图片中的文字，而 VLM 能理解整个场景的含义（比如看到冰箱里的食材后推荐菜谱）。这是 VLM 比传统计算机视觉更强大的地方。

跨越模态：JSON 请求体的进化

单模态：纯文本时代

以前，我们和模型的交流仅限于一句话：

```
messages=[
  {
    "role": "user",
    "content": "请写一首关于春天的诗" # 纯字符串
  }
]
```

多模态：混合指令集

现在，`content` 变成了一个数组，可以同时装文本和图片：

```
messages=[
  {
    "role": "user",
    "content": [
      # 文本指令
      {"type": "text", "text": "描述这张图片"},
      # 图片数据
      {"type": "image_url", "image_url": {
        "url": "https://example.com/cat.jpg"
      }}
    ]
  }
]
```

[解释]

content 字段从字符串变成数组

这是本讲最核心的技术变化。在纯文本模式下，`messages` 里的 `content` 就是一个普通字符串。而在多模态模式下，`content` 变成了一个数组（列表），里面可以同时放文本和图片。每个元素用 `type` 字段标识类型：`text` 表示文本指令，`image_url` 表示图片数据。

图片传输的两种方式

方式一：公网 URL

直接传一个 `http://...` 地址，让模型服务器自己去下载图片。

- **优点：**代码简洁，不占本地内存。
- **局限：**模型服务器必须能访问到这个地址。
如果传的是本地 `127.0.0.1/1.jpg`，云端模型抓不到图。

方式二：Base64 编码

将图片的二进制数据编码为 ASCII 字符串，直接嵌入请求体中。

- **格式：**
`data:image/jpeg;base64,/9j/4AAQSk...`
- **优势：**自带图片数据，不受网络限制，最适合处理用户刚上传的本地图片。

如何选择？

- 图片在公网上且 URL 稳定 → 用 URL，省带宽。
- 图片在用户本地或需要保护隐私 → 用 Base64，自包含传输。

本讲后续的黑客松实战中，我们将使用 **Base64** 方式，因为题目图片来自平台接口，需要先下载到本地再编码发送。

[解释]

URL 与 Base64 的选择策略

图片传给视觉模型有两种方式。URL 方式代码简洁，但要求图片地址公网可达；Base64 方式把图片编码为一串文本嵌入请求体，不依赖外部网络，适合处理本地图片或需要保护隐私的场景。

在实际工程中，选择取决于图片来源：如果是公网上的稳定图片（如商品图），用 URL 省带宽；如果是用户刚上传的本地文件，用 Base64 更可靠。本讲后续的黑客松实战采用 Base64，因为题目图片需要先下载到本地。

Step 1: 调用 Qwen3-VL 识别本地图片

Python 视角：手动编码 Base64

```
import base64
from openai import OpenAI

# 1. 局部工具函数：编码器（将本地图片转为文本）
def encode_image(image_path):
    with open(image_path, "rb") as image_file:
        return base64.b64encode(image_file.read()).decode('utf-8')

# 📁 替换为自己的图片路径
base64_img = encode_image("sample.jpg")

# 2. 组装多模态消息体
messages = [{
    "role": "user",
    "content": [
        {"type": "text", "text": "请用中文详细描述图片内容。"},
        {"type": "image_url", "image_url": {
            # 拼装为标准 Data URI
            "url": f"data:image/jpeg;base64,{base64_img}"
        }}
    ]
}]
```

调用视觉模型

```
client = OpenAI(...)

response = client.chat.completions.create(
    # 指定通义千问最强视觉开源模型
    model="Qwen/Qwen3-VL-8B-Instruct",
    messages=messages
)

print(response.choices[0].message.content)
```

显著简化：在过去，让程序识别图片需要配置复杂的 OpenCV 环境和张量矩阵。现在，只需会拼装 JSON 请求即可完成。

[解释]

Base64 编码与 VLM 调用流程

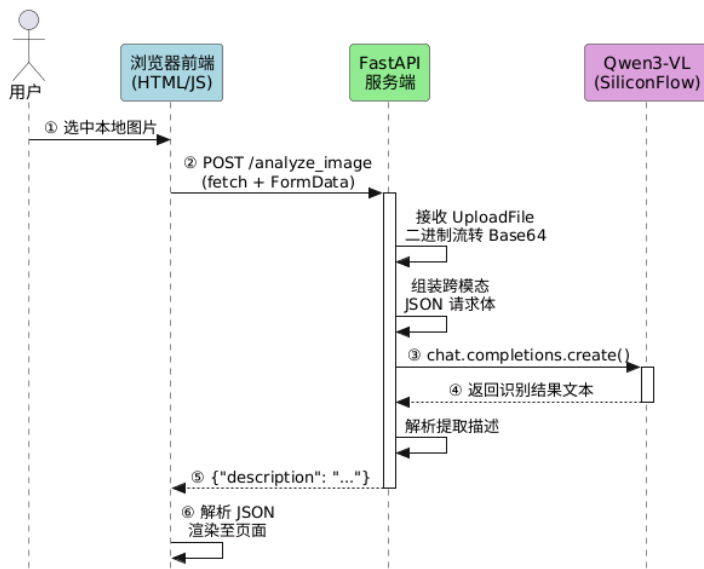
本页展示的是完整的本地图片识别代码。核心流程分两步：第一步，`encode_image` 函数把本地图片文件读出二进制数据，然后转为 Base64 字符串；第二步，把这串文本按 `data:image/jpeg;base64,...` 格式填入 `content` 数组中的图片位置。

这段代码和上周学的纯文本调用相比，只多了两个变化：(1) 多了一个编码函数；(2) `content` 从字符串变成了数组。其余的 SDK 调用方式完全一样，这就是统一接口的好处。

Step 2: 用 FastAPI 封装为 Web 应用

前后端交互的宏观数据流

我们将上周掌握的 FastAPI 中间件（L22），与刚才的视觉 API 进行整合。



为什么需要一层后端？

为何不让浏览器前端的 JS 直接拿着 Base64 去调大模型？

1. **严重的安全隐患**：JS 代码完全暴露在客户端。如果直接请求，`SILICONFLOW_KEY` 将直接明文写在网页里，任何人按 F12 就能盗用余额。这不是假设——据 GitGuardian 报告，仅 2023 年 GitHub 上就检测到 1280 万个泄露的密钥，其中 OpenAI API Key 泄露量激增。
2. **跨域墙 (CORS)**：大量模型服务商禁止浏览器跨域直连。

注意：始终在受控的服务器端（FastAPI）隐藏 API Key，浏览器前端只负责纯粹的渲染工作。

[解释]

前后端分离与 API Key 安全

本页的核心知识点是：为什么不能让浏览器直接调用大模型。原因很简单——浏览器里的 JS 代码是完全公开的，任何人按 F12 都能看到。如果把 API Key 写在前端，等于把钥匙挂在门外。

解决方案是加一层 FastAPI 后端。浏览器只负责显示界面和发送图片，FastAPI 服务器负责在安全环境里贴上 API Key、调用模型、再把结果返回给浏览器。这种架构在上周 L22 学过的 FastAPI 基础上直接复用，只是多了一个图片接收和 Base64 编码的步骤。

Step 2 续：驱动大模型代工全栈代码

写界面的体力活，交给 AI 助手。我们只需构建 HOTL 约束指令。

四大架构约束（不可缺漏）

1. **前后端统筹**：要求返回 HTML 或使用 Jinja2 模板。
2. **通信契约**：明确要求 `/analyze_image` 接收 `UploadFile`。
3. **大模型黑盒对接**：锁死使用 `openai` SDK，指明 `Base_url` 和模型名称。
4. **UX（用户体验）**：必须带 Loading 动画，缓解大模型分析时的漫长等待感。

使用以下 Prompt 生成产品原型

"请帮我写一个基于 FastAPI 的多模态看图应用（前后端合并在一个文件，通过返回 HTML 字符串驱动）。

核心要求：

1. 提供精美的暗黑模式网页，包含一个图片上传按钮和一个结果展示框。
2. 提供 `/analyze_image` 接口，接收图片流，并将其转为 Base64 编码。
3. 在接口内部，使用 `openai` 库（`base_url` 为 `https://api.siliconflow.cn/v1`）调用 `Qwen/Qwen3-VL-8B-Instruct`，要求它用中文详细描述图片内容。
4. **注重体验**：前端上传期间必须有 Loading 状态展示。API Key 从系统环境变量读取。"

[解释]

HOTL 约束指令与 AI 代工

本页展示了一种重要的工作模式：用 AI 生成代码，但人类制定架构约束。Prompt 中卡死了 4 个维度：前后端统筹、通信契约、大模型对接、用户体验。这确保了 AI 生成的代码能符合整体架构要求，而不是随意发挥。这个工作模式就是课程中反复强调的 HOTL（Human-On-The-Loop）：人类不写具体代码，但始终掌控架构决策和质量标准。

Step 3: 黑客松谜题挑战——VLM 自动解题

脱离 UI，走向纯粹的 API 智能体

真实的智能体不需要图形界面，它们在后台通过 API 与世界交互。

本次黑客松挑战，要求你编写一个 Python 脚本，完成全自动的“获取题目 -> 看图解题 -> 提交答案”闭环。

竞技流程要求：

1. 获取凭证：登录

`https://syb.ncu.edu.cn/smart-class/apikey` 获取专属 API_KEY。

2. 感知 (拉取题目)：请求系统 API 获取专属谜题图片 URL 与全局唯一的 `question_id`。

3. 推理 (VLM 解析)：调用大模型看懂算术图片，计算出数字答案。

4. 行动 (提交结果)：将答案与 `question_id` 自动 POST 传回判题服务器。

核心闭环代码骨架

```
import requests
from openai import OpenAI

API_KEY = "你的_API_KEY"
client = OpenAI(...)

# 1. 感知：向平台拉取专属题目
puzzle_url = f"https://syb.ncu.edu.cn/smart-class/api/hackathon/vision/puzzle?api_key={API_KEY}"
puzzle_res = requests.get(puzzle_url).json()
q_id = puzzle_res["question_id"]
img_url = puzzle_res["image_url"]

# 2. 推理：VLM 视觉解题 (公网 URL 无需 Base64)
resp = client.chat.completions.create(
    model="Qwen/Qwen3-VL-8B-Instruct",
    messages=[
        {
            "role": "user",
            "content": [
                {
                    "type": "text", "text": "图中是一道算术题。请只返回计算结果的数字。",
                },
                {
                    "type": "image_url", "image_url": {"url": img_url}
                }
            ]
        }
    ]
)
answer = resp.choices[0].message.content.strip()

# 3. 行动：自动提交答案
submit_url = "https://syb.ncu.edu.cn/smart-class/api/hackathon/vision/answer"
submit_res = requests.post(
    submit_url,
    headers={"X-API-Key": API_KEY, "Content-Type": "application/json"},
    json={"question_id": q_id, "answer": answer}
)
print("通关反馈:", submit_res.json())
```

这就是完整的 AI 工程闭环：感知（网络获取题目并看图）→ 推理（逻辑计算）→ 行动（自动提交结果）。这个纯代码构成的自动机，就是下周黑客松项目的核心骨架。

[解释]

脱离 UI 的 API 智能体闭环

Step 2 展示了带有 UI 界面的 Web 服务。但在工业界，大量智能体运行在后台（如爬虫、自动化脚本），通过 API 与外界通信。Step 3 抛弃了前端界面，直接通过 Python 的 `requests` 库完成了获取题目和提交答案的操作。这展示了智能体“纯后台自动化运行”的工程形态。

感知-推理-行动 (Perception-Reasoning-Action)

这是智能体 (Agent) 的最基本架构模式。本步骤中：

- **感知**：程序读取 API 获取环境信息（包含题目的 URL），VLM 理解视觉内容。
 - **推理**：模型在内部完成数学计算。
 - **行动**：程序将计算得出的答案通过 HTTP POST 提交给判题系统，改变了外部环境状态。
- 这个三步闭环，就是 L27-L28 将要深入讲解的 Agent 架构雏形。

四、小结

感知-推理-行动：最小闭环的拼图完成

[解释]

章节定位：知识收敛

本章对全讲内容做归纳总结，回顾感知-推理-行动闭环的完整实现，并预告下一讲的 Pipeline 流水线架构。

总结与预告

完成智能体最重要的一跃

今天我们完成了从纯文本到多模态的技术升级，更重要的是，我们在代码架构上落地了“感知-推理-行动”闭环：

- **感知**：通过 Base64 和公网 URL 两种方式，让程序能够接收并理解视觉图像。
- **推理**：通过 OpenAI SDK 封装底层通信，让我们可以专注于如何用大模型完成逻辑推理。
- **行动**：通过后端主动发起 POST 请求，让程序能够自主地向第三方平台提交结果。

向 Agent (智能体) 进军

1. **核心简化**：在多模态生态中，图像被编码为 Base64 文本。用处理文本的思维即可调度视觉模型，无需学习底层算法。
2. **感官的可插拔扩展**：今天我们只是调用 VLM 给智能体装上了“眼睛”。基于这套标准的闭环架构，未来只要换成 STT（语音转文本）API 就能赋予智能体“听觉”，或是接入 IoT 传感器 API 赋予它更丰富的物理环境感知。
3. **下一讲预告**：解决了“感知”层后，我们要挑战更复杂的业务场景。下一课，我们将学习 **Pipeline (流水线) 模式**，让多个模型串联接力，从而大幅增强智能体的核心“推理”能力。

结语：尽管这只是一个几十行代码的课堂练习，但我们已经完整走通了“感知 - 推理 - 行动”的闭环，亲手实现了一个微型的智能体。技术持续在迭代，但这种架构思维，才是最重要的可迁移能力。

[解释]

本讲知识体系回顾

本讲完成了三个技术升级：(1) 从 requests 到 SDK 的抽象跃升；(2) 从纯文本到多模态的感知延展；(3) 从单个脚本到具备行动能力（POST 提交）的闭环应用。

向教师强调：在总结时务必点出“感知层是可插拔的 (Pluggable)”这一架构思想。今天是视觉，明天就可以是听觉 (STT) 或物联网传感器。下一讲 (L26) 将在这个闭环基础上，引入多模型 Pipeline 流水线，重点增强智能体的“推理 (Reasoning)”能力。